

AEGIS

A Post-Quantum Secure Communications Overlay
for Transport-Independent, Disconnection-Tolerant Environments

SYSTEM DOCUMENTATION MANUAL

Smart India Hackathon 2026

Prepared for technical evaluation and judge review

Table of Contents

1. What Is Aegis?	3
2. Why We Built Aegis	4
3. What Aegis Is — and Is Not	6
4. System Architecture Overview	8
5. The Cryptographic Stack	10
6. The Hybrid Handshake	11
7. Authentication and Identity	14
8. End-to-End Messaging Flow	16
9. The Double Ratchet	17
10. State Persistence — Surviving an App Restart	19
11. Transport Independence	21
12. LAN Operation	23
13. Disconnected / Degraded Operation	24
14. What the Server Can and Cannot See	25
15. Threat Model	26
16. Why These Specific Technologies	28
17. The Post-Quantum Packet-Size Problem	30
18. Deployment Scenarios	32
19. Known Gaps and Roadmap	34
20. Judge FAQ	36
21. Glossary	39
22. Team & Contributors	40

1. What Is Aegis?

Aegis is a post-quantum secure communications overlay: a cryptographic layer that protects the confidentiality, integrity, and authenticity of messages regardless of the network the message happens to travel over, and regardless of whether that network is available, degraded, or momentarily absent.

It is not, at its core, a chat application. The chat client that judges will see in the demonstration is a client of the Aegis layer, not the product itself. The product is the security layer underneath it: a hybrid post-quantum handshake, an authenticated identity system, a self-updating message-key ratchet, and a transport abstraction that lets the exact same cryptographic session continue over the internet, over a local network, or after a period of complete disconnection.

In one sentence

Aegis lets two endpoints establish and maintain a quantum-resistant encrypted session whose security does not depend on any particular network path, and whose relay infrastructure never possesses the material required to read the messages it carries.

1.1 Why the distinction matters

Several existing products already do ‘post-quantum encrypted chat’ competently — Signal has deployed a hybrid post-quantum handshake (PQXDH) in production, and dedicated products such as PQCChat offer browser-based post-quantum messaging with ML-KEM and ML-DSA. Building another browser chat app with the same primitives would not be a meaningful engineering contribution; it would be a re-implementation of a solved problem.

Aegis instead targets the harder, less-solved problem sitting underneath all of those products: every mainstream secure-messaging system quietly assumes that a central server is reachable. Signal, WhatsApp, PQCChat, and most enterprise messengers fetch the recipient's public key material from a server at the start of a conversation and rely on that server to relay traffic afterward. That assumption is reasonable for a consumer internet product. It breaks down in disaster response, rural deployment, and any environment where connectivity is intermittent rather than continuous.

Aegis is built around removing that assumption — without weakening the cryptography, and without pretending to be something it is not (see Section 3 and Section 15 for an honest discussion of what is, and is not, currently demonstrated).

2. Why We Built Aegis

2.1 Problem One — Harvest Now, Decrypt Later

Modern secure messaging depends heavily on classical public-key cryptography — RSA, elliptic-curve Diffie-Hellman, and related constructions. Their security rests on mathematical problems (integer factorization, discrete logarithms) that are hard for classical computers but are known to be efficiently solvable by a sufficiently large, fault-tolerant quantum computer running Shor's algorithm.

Such a machine does not exist today. But an adversary does not need it today. A passive attacker can record encrypted traffic now, store it indefinitely, and decrypt it retroactively once capable quantum hardware exists. This is known as Harvest Now, Decrypt Later (HNDL). It only matters for information that needs to stay confidential for years — which describes most government, defense, medical, and legal communication, and most disaster-response coordination that later becomes part of an official record.

Why this is not paranoia

The threat only requires that (a) an adversary can capture ciphertext today, and (b) the plaintext still matters in five, ten, or twenty years. Both conditions are already true for the communication categories Aegis targets, independent of when or whether a cryptographically relevant quantum computer is ever built.

2.2 Problem Two — Network Dependency

The second, less discussed problem is architectural, not cryptographic. Almost all secure messengers are built around a single connectivity assumption:



In disaster zones, rural regions, temporary command posts, or any tactically constrained environment, that path is frequently unavailable, intermittent, or actively unreliable, even while two devices remain on the same local network. A messaging system whose security model quietly collapses the moment the internet disappears is unsuitable for exactly the situations where secure communication matters most.

2.3 Problem Three — Server Trust

Even when a server is reachable, it represents a single point of compromise: it can be hacked, misconfigured, legally compelled, or operated by a party the users should not have to trust with plaintext. Reducing what a compromised or coerced server can reveal is therefore a design goal in its own right, independent of the quantum and connectivity problems.

2.4 The synthesis

Aegis treats these as one design problem, not three separate features bolted together:

1. Cryptography must resist both classical and (anticipated) quantum adversaries — addressed with hybrid key establishment (Section 6).
2. The security layer must not care which network carries its bytes — addressed with transport independence (Section 10).
3. The relay infrastructure must be structurally incapable of reading message content — addressed with the blind-relay architecture (Section 11).

3. What Aegis Is — and Is Not

3.1 Honest competitive positioning

It is important to state plainly what already exists, because judges familiar with the space will ask. Signal has shipped PQXDH, a hybrid X25519 / post-quantum KEM handshake, specifically to defend against harvest-now-decrypt-later, and has publicly discussed a subsequent post-quantum ratchet for ongoing forward secrecy. PQChat offers a working browser-based post-quantum messenger with ML-KEM and ML-DSA. Other products (e.g. Polynom) target post-quantum enterprise collaboration suites.

Aegis does not claim to have invented post-quantum messaging, and does not compete with those products on chat features, user experience, or ecosystem maturity. It would lose that comparison, and pretending otherwise in front of technically literate judges would be a mistake.

3.2 What Aegis actually claims

The actual claim

Aegis is a prototype cryptographic communications overlay that keeps a single hybrid post-quantum session alive and secure across multiple, independently-selectable transport paths — including a direct local-network path used when no central infrastructure is reachable at all — while keeping the relay server structurally blind to message content. The chat interface exists to demonstrate the overlay; it is not the contribution.

This claim has three parts, each independently demonstrable:

- Hybrid post-quantum security — the same well-studied primitives other products use, composed carefully (Section 6).
- Demonstrated transport independence — the identical cryptographic session and packet format running over more than one transport, with the crypto engine unaware of which one is active (Section 10).
- A blind relay — verifiable by inspecting server storage and finding no plaintext or session secrets (Section 11).

3.3 What Aegis is deliberately not claiming

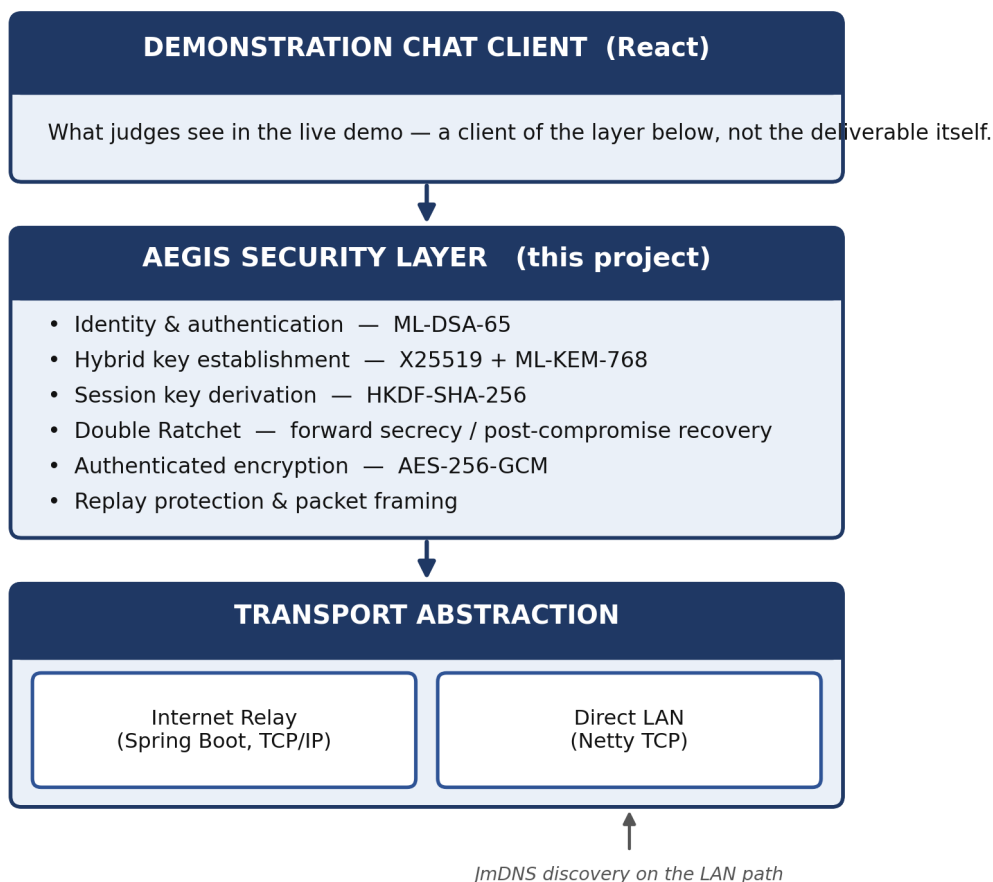
To keep the project defensible under technical questioning, the team explicitly avoids these claims:

- “Certified or integrated with a specific military radio, waveform, or classified network.” The prototype defines a transport interface and demonstrates it over generic bearers (TCP, LAN, and a simulated constrained link); integrating a specific fielded radio is future work, not a completed deliverable.
- “A solved military key-management system.” Real operational key management includes credential issuance, device enrollment, revocation, escrow policy, and classification-domain separation — far beyond what a six-month student prototype can responsibly claim. Aegis implements an identity and session-key architecture appropriate to a prototype, described honestly in Section 8 and Section 16.

- “Unhackable”, “quantum-proof”, or “100% secure.” These are not technically meaningful claims and are avoided throughout this document (see Section 18).
- “Ready for military deployment.” The realistic pathway from a hackathon prototype to a fielded system runs through security review, protocol audit, penetration testing, formal requirements, and procurement — a multi-year institutional process, not a demo weekend (Section 17).

4. System Architecture Overview

Aegis is organized as three layers. The bottom two layers are the actual engineering contribution; the top layer is the demonstration surface.



The Aegis security layer never inspects or depends on which transport is active. It hands the transport layer opaque encrypted packets and expects opaque encrypted packets back. This separation is the single most important architectural decision in the project and is discussed in depth in Section 10.

4.1 Major components

Component	Role	Owns / sees
Client Crypto Engine	Runs on the user's device. Generates keys, performs the handshake, manages ratchet state, encrypts/decrypts.	Private identity key, root key, chain keys, message keys — never leaves the device.
React UI (demo client)	The chat interface used in the demonstration.	Plaintext for display only; delegates everything cryptographic to the engine.
Electron shell (LAN builds)	Gives the client access to raw TCP sockets	No cryptographic material of its own;

Component	Role	Owns / sees
	and multicast DNS, which a browser sandbox does not permit.	a native-capability host for the crypto engine.
Spring Boot Relay	Account auth, session tokens, public pre-key directory, ciphertext relay, temporary offline queue.	Public keys, signatures, ciphertext, metadata. Never plaintext or private/session keys.
Netty TCP	Moves encrypted bytes directly between two peers on the same local network.	Nothing cryptographic — operates purely on opaque encrypted packets.
JmDNS	Discovers peer presence, address, and port on the local network.	Network-level presence only. Explicitly not an authentication mechanism (Section 8.4).

5. The Cryptographic Stack

Aegis follows one governing rule: never invent a new cryptographic primitive. Every algorithm below is a standardized, publicly reviewed construction. The engineering work is in composing them correctly, not in designing new mathematics.

Purpose	Algorithm	Notes
Post-quantum key establishment	ML-KEM-768	FIPS 203. Lattice-based (Module-LWE). Establishes a shared secret; does not encrypt the message itself.
Classical key establishment	X25519	Elliptic-curve Diffie-Hellman. Provides the classical half of the hybrid handshake.
Authentication / signatures	ML-DSA-65	FIPS 204. Signs identity and pre-key bundles so a recipient can verify who a public key claims to belong to.
Key derivation	HKDF-SHA-256	Combines the classical and post-quantum secrets into a root key and derives further session sub-keys.
Message encryption	AES-256-GCM	Authenticated encryption for the actual message payload — confidentiality and integrity together.
Forward secrecy / post-compromise recovery	Double Ratchet	Continuously replaces message keys and periodically re-runs a Diffie-Hellman step to recover security after a temporary compromise.

A phrase to avoid

“Combining two algorithms automatically makes the system secure.” It does not, by itself. Security also depends on correct composition, mutual authentication, transcript binding, downgrade resistance, sound implementation, and disciplined key lifecycle management. Each of those is addressed explicitly in the sections that follow, and the ones not yet fully closed are named honestly in Section 16.

6. The Hybrid Handshake

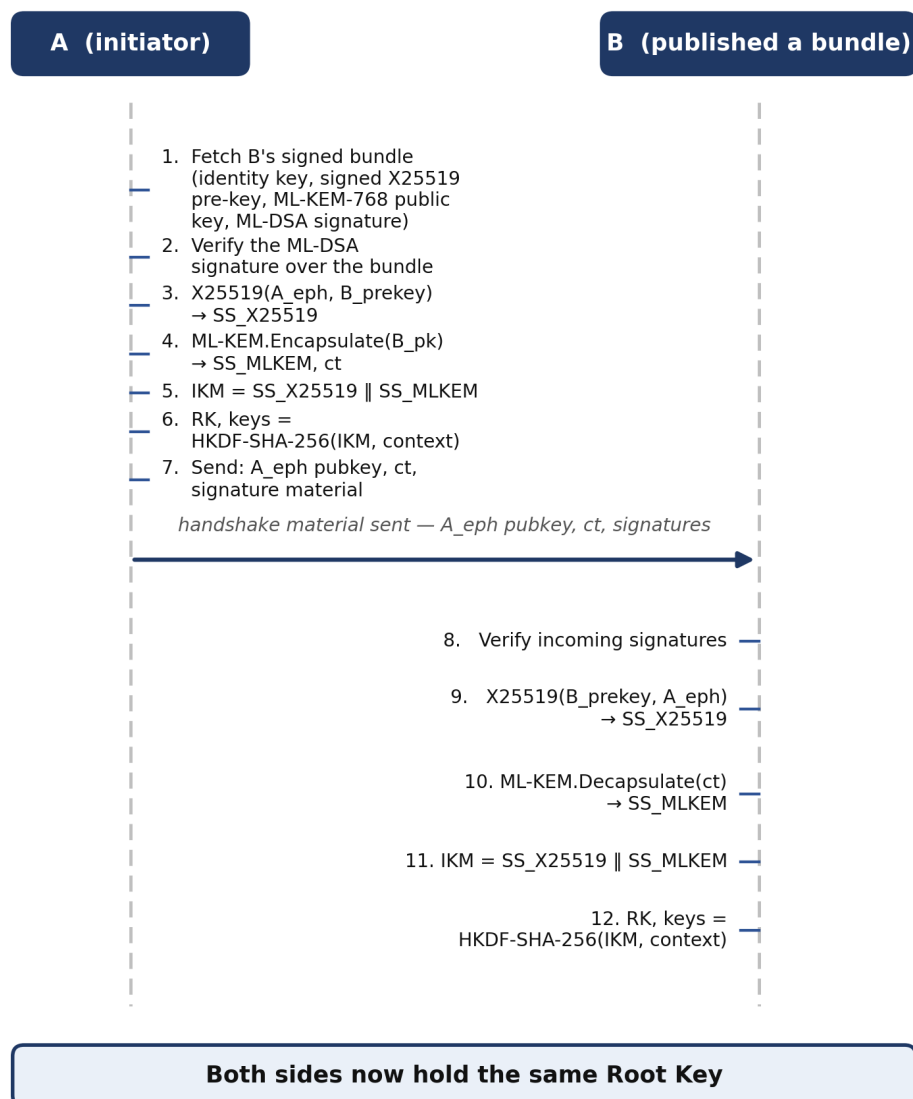
6.1 Why hybrid, specifically

Aegis derives its session secret from two independent key-establishment mechanisms rather than one:

- X25519 — classical elliptic-curve Diffie-Hellman, well understood, fast, and small, but breakable by a sufficiently capable quantum computer running Shor's algorithm.
- ML-KEM-768 — a lattice-based post-quantum key encapsulation mechanism believed to resist known quantum attacks, standardized by NIST as FIPS 203.

The two are combined so that an attacker must break both mechanisms to recover the session secret. If ML-KEM turns out to have an unforeseen weakness (post-quantum cryptography is comparatively young), X25519 still contributes real classical security. If a quantum computer eventually breaks X25519, ML-KEM is designed to remain resistant under its own assumptions. Neither component is trusted alone.

6.2 The construction



The shared secret itself is never transmitted. ML-KEM produces a ciphertext that only the holder of the matching private key can decapsulate into the same secret — the ciphertext is not the secret, and intercepting it does not reveal the secret.

6.3 Four requirements that make the combination actually secure

Selecting X25519 and ML-KEM is the easy part. Four additional properties are what make the composition trustworthy, and each is treated as a first-class requirement rather than an afterthought:

Fixed concatenation order

Both sides must concatenate the two secrets in exactly the same order before hashing (`SS_X25519 || SS_MLKEM`, never the reverse on one side). A mismatch silently produces two different root keys and the session fails closed — which is safe, but must be tested for explicitly rather than discovered in the field.

Context binding in HKDF

The HKDF step does not run on raw key material alone. Its context/info parameter binds the derived key to a protocol version identifier, the algorithm identifiers actually negotiated, and identity/session information from both sides. This is what provides domain separation and helps prevent an attacker from confusing one protocol run for another, or quietly forcing a downgrade to weaker parameters. Demonstration or textbook HKDF “info” strings from early research notes are treated as placeholders, not production values, until independently reviewed.

Downgrade resistance

The negotiated algorithm set is itself covered by the transcript that gets authenticated. An active attacker who tries to strip the post-quantum component and force a classical-only exchange must be detectable by both endpoints, not silently accepted.

Key confirmation

After deriving the root key, both sides derive a short confirmation value from it and compare them before treating the session as live. This catches wrong key combination, transcript mismatch, or tampering during the handshake itself, rather than surfacing as silently undecryptable messages later.

Key establishment ≠ authentication

ML-KEM guarantees that two parties who complete the protocol correctly arrive at the same secret. It says nothing about whether the party on the other end is actually who they claim to be. That is a separate problem, solved by ML-DSA signatures and an identity-trust model (Section 7) — not by the KEM.

7. Authentication and Identity

7.1 Identity keys

Every Aegis user holds a long-term ML-DSA-65 identity key pair. The private half never leaves the originating device. The public half becomes that user's cryptographic identity and is used to sign pre-key bundles.

7.2 Pre-key bundles

To allow a session to start even when the recipient is offline (asynchronous session establishment), each user publishes a signed bundle in advance:

- Identity public key (ML-DSA-65)
- Signed X25519 pre-key
- ML-KEM-768 public key
- One or more one-time X25519 pre-keys (optional, improves per-session forward secrecy)
- ML-DSA signature over the bundle

A sender verifies the signature before using any of the bundle's key material. An invalid signature means the bundle is rejected outright — the handshake does not proceed on unverified key material under any circumstance.

7.3 The trust-anchor problem, stated honestly

ML-DSA signatures prove that a bundle was signed by whichever identity key is attached to it. They do not, by themselves, prove that the identity key belongs to the person the user thinks it belongs to. If a malicious or compromised relay server substitutes an attacker's entire identity key and re-signs a fresh bundle under that key, a client with no prior information about the real key has no cryptographic way to detect the substitution on the very first contact. This is the same trust-anchor problem every system in this space has (Signal, SSH, and PGP all face a version of it), and Aegis does not claim to have solved it in a way nobody else has — it adopts the same practical mitigation these systems use.

7.4 The Aegis trust model: TOFU + optional out-of-band verification

1. First contact — Alice records Bob's ML-DSA identity fingerprint the first time she receives it (Trust On First Use, as in SSH).
2. Key-change detection — if Bob's identity key ever changes on a later contact, Alice's client raises a prominent warning instead of silently accepting the new key.
3. High-assurance mode — Alice and Bob can compare a short fingerprint or QR code over a separate trusted channel (in person, over a verified voice call) to remove reliance on TOFU entirely.
4. Disaster / no-internet contact — if Bob is discovered over the LAN via JmDNS, his bundle can be exchanged directly, but JmDNS discovery still carries no trust weight of its own. The fingerprint that

was previously verified (or the TOFU record) is what determines whether the discovered key is trusted, not the fact that it was found on the local network.

Status of this section

TOFU and fingerprint/QR verification are the design decision, not yet a fully implemented and tested feature at the time of writing. This document describes the trust mechanism Aegis is built around; the phase plan in Section 19 tracks what is implemented versus planned.

8. End-to-End Messaging Flow

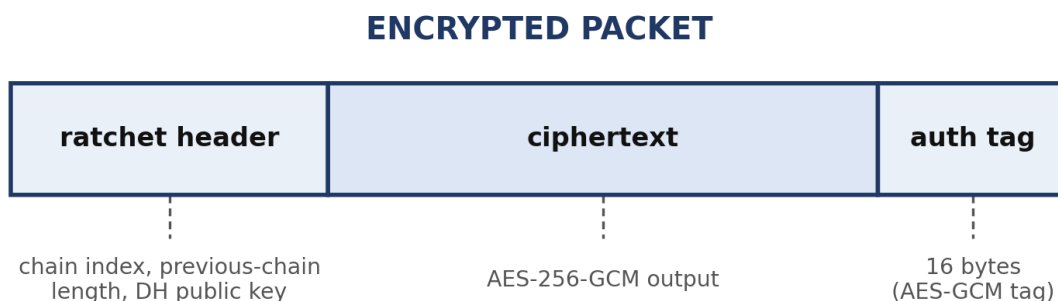
The complete lifecycle of a conversation, from identity creation to a delivered message:

8.1 Step by step

1. Identity creation — Bob generates his ML-DSA identity key pair locally. The private key never leaves the device.
2. Bundle publication — Bob creates a signed pre-key bundle (Section 7.2) and uploads only the public parts to the relay server.
3. Discovery — Alice requests Bob's bundle from the server (or, on a LAN with no server reachable, discovers Bob directly via JmDNS).
4. Verification — Alice verifies Bob's ML-DSA signature over the bundle before using any key inside it (Section 7).
5. Hybrid handshake — Alice performs X25519 and ML-KEM encapsulation against Bob's published keys and derives a root key via HKDF (Section 6).
6. Key confirmation — both sides derive and compare a confirmation value; the session is only considered live on a match.
7. Ratchet initialization — the root key seeds the Double Ratchet's sending and receiving chains (Section 9).
8. Message encryption — each message is encrypted with a fresh, ratchet-derived key using AES-256-GCM.
9. Transport selection — the encrypted packet is handed to whichever transport is currently active — internet relay or direct LAN (Section 10).
10. Delivery and decryption — the recipient's ratchet derives the matching key from the packet's header and decrypts and authenticates the ciphertext.

8.2 What the transport layer actually sees

Regardless of path, the object moving over the network is the same:



Nothing in this packet requires a decryption decision by the transport. Netty, the relay server, and any future bearer only ever move this structure as opaque bytes.

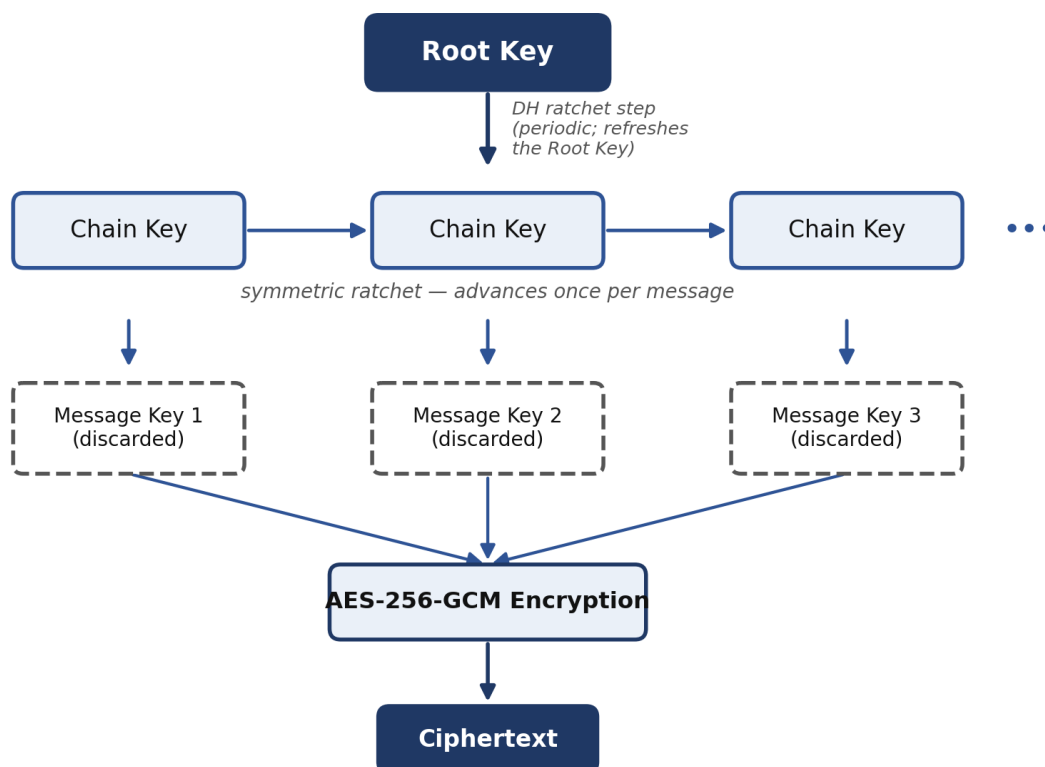
9. The Double Ratchet

9.1 Why not one static session key

If a single symmetric key encrypted every message for the life of a session, compromising that one key at any point would expose every past and future message encrypted with it. The Double Ratchet exists to make that single point of failure disappear.

9.2 Two ratchets, two jobs

Ratchet	Advances on	Provides
Symmetric (chain) ratchet	Every message sent or received	A fresh, unique message key per message; old message keys are discarded immediately after use.
Diffie-Hellman ratchet	Periodically, when a new DH public key is received from the peer	Refreshes the root key itself, so that even long-term compromise of the chain state is gradually 'healed' as the conversation continues.



9.3 What this buys, and what it does not

- Forward secrecy — compromising today's key material does not expose yesterday's messages, because yesterday's keys have already been discarded.
- Post-compromise recovery — after a temporary compromise ends, the periodic DH ratchet step reintroduces fresh secret material the attacker never saw, gradually restoring confidentiality for future messages.

What the ratchet cannot do

The Double Ratchet does not make a fully compromised, currently-active device safe. If an attacker has live control of an endpoint, they can read plaintext, observe active keys, and continue to do so for as long as the compromise persists — no ratchet design changes that. The correct claim is narrower and more defensible: the ratchet limits the blast radius of a compromise in time, it does not eliminate the consequences of an active one.

9.4 Practical ratchet edge cases

A production-quality ratchet implementation has to handle, not just describe, the following:

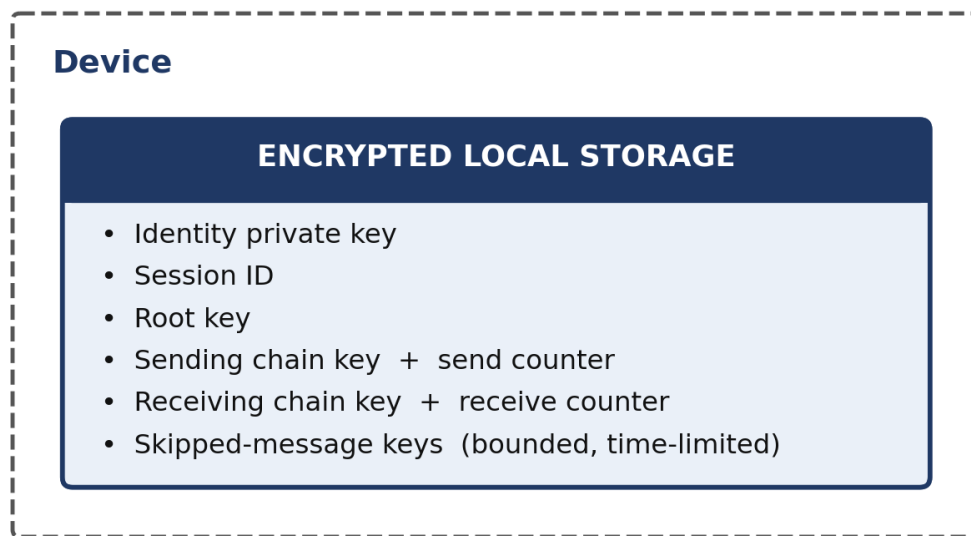
- Out-of-order delivery — message N+1 can legitimately arrive before message N on an unreliable transport; the receiver must be able to derive keys out of strict order.
- Skipped messages — if a message is lost entirely, its message key must be cached for a bounded period so a late/retransmitted copy can still be decrypted, then eventually discarded.
- Duplicate delivery — replay of an already-processed packet must be detected and dropped rather than re-decrypted or re-displayed (Section 14).
- Simultaneous initiation — both sides starting a session at the same time needs a deterministic tie-breaking rule so they converge on one session, not two.

10. State Persistence — Surviving an App Restart

A common and fair question: the ratchet state (root key, chain keys, message counters, skipped-message keys) clearly has to exist somewhere while the app is running. What happens when the app is closed?

10.1 The design

Ratchet state is not RAM-only. It is written to encrypted local storage on the device, with the storage's own encryption key protected by the operating system's secure key-storage facility where one is available (e.g. platform keychain / secure enclave equivalents).



On restart: decrypt local storage, restore root key / chain keys / counters, and resume the ratchet from exactly that point. No new handshake is performed — a new handshake is only triggered if the session itself is invalidated (e.g. identity-key change, explicit logout, or an unrecoverable state error).

10.2 The failure mode this must avoid

Atomic advancement is mandatory

The ratchet state must be persisted atomically as it advances. If a crash occurs between 'decrypt an incoming message' and 'save the resulting new chain key,' the state can roll back to a previous point and desynchronize from the peer's state, breaking future decryption. The implementation must treat 'process message' and 'persist new state' as a single atomic unit, not two separate steps that can be interrupted independently.

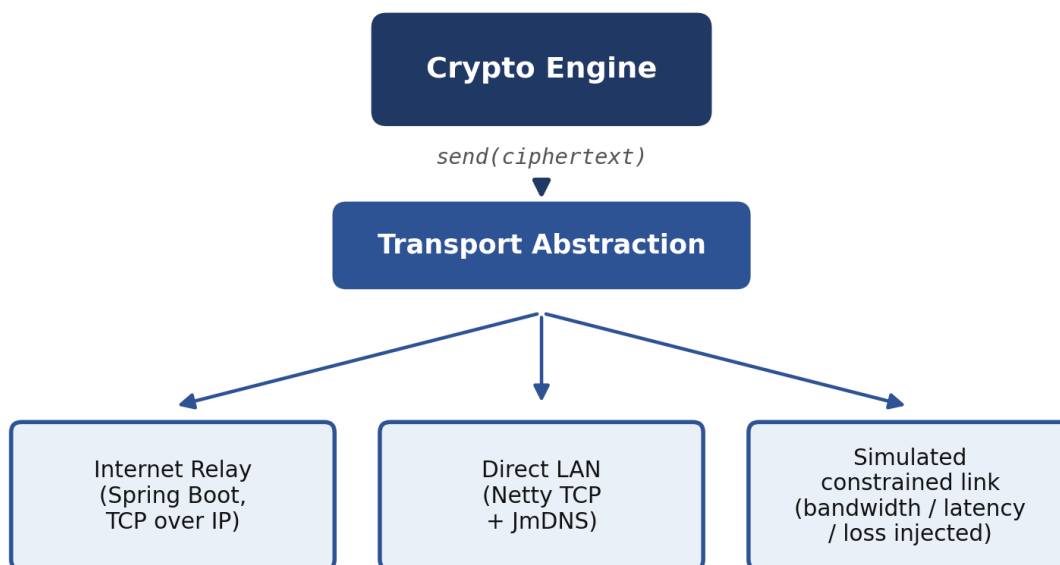
10.3 Status

Encrypted-at-rest persistence with OS-backed key storage is the specified design; it is implementation work still to be completed and tested, not a feature already demonstrated. It is listed as a tracked item in Section 19.

11. Transport Independence

11.1 The principle

The Aegis crypto engine produces one thing: an encrypted packet. It hands that packet to a transport abstraction and does not know or care which concrete transport is underneath. This is the architectural property the project is actually trying to prove, and it is only meaningful if it is demonstrated, not just diagrammed.



Note what the crypto engine does not see: it never calls anything resembling `send_over_wifi()` or `send_over_relay()`. It calls `send(ciphertext)`, and a transport manager underneath decides how that gets moved. Proving this requires showing the identical client, session, and packet format running over at least two materially different transports — not merely asserting that the architecture diagram allows it.

11.2 Network handover in practice

Consider the demonstration scenario: Alice and Bob are chatting through the internet relay. The router the relay depends on fails, but Alice and Bob remain on the same Wi-Fi/LAN segment.

1. Alice's transport manager detects that the relay connection has failed (connection error / heartbeat timeout).
2. JmDNS resolves Bob's presence and address on the local network.
3. A direct Netty TCP connection is established to Bob.
4. The next outgoing packet — same ratchet state, same header format, same ciphertext structure — is sent over that connection instead.

5. Bob's crypto engine processes the ratchet header exactly as it would have over the relay. No renegotiation, no new handshake.

Honest scope note

Detecting relay failure, discovering the peer, and switching transports automatically and seamlessly (with no error surfaced to the user) is application/transport-manager logic that must be built and tested — it is not automatic just because the crypto layer is transport-agnostic. What the architecture guarantees is that the ratchet sequence and cryptographic state remain valid across the switch; it does not by itself guarantee an instant, invisible handover without additional engineering in the transport manager.

12. LAN Operation

The LAN path exists so that communication continues functioning when the internet is unavailable but a local network is not — for example, two laptops on the same router with no upstream internet connection.

12.1 JmDNS — discovery, not authentication

JmDNS (multicast DNS service discovery) lets devices on the same local network announce and find each other: presence, network address, and service/port information. This is the single most important caveat in the entire LAN design:

Discovery is not authentication

Finding a device advertising itself as “Bob's Laptop” on the local network does not prove that device belongs to Bob, or that it is not an attacker's machine impersonating Bob's service name. JmDNS answers the question ‘where is a device claiming to be Bob?’ It never answers ‘is this actually Bob?’ — that question is answered exclusively by the cryptographic identity verification in Section 7.

12.2 Netty — transport, not trust

Netty provides the TCP connection used once a peer has been located. It moves bytes; it does not make decisions about who to trust, and it does not contain decryption or key-management logic of any kind. Everything it sees is already ciphertext.

12.3 Why Electron

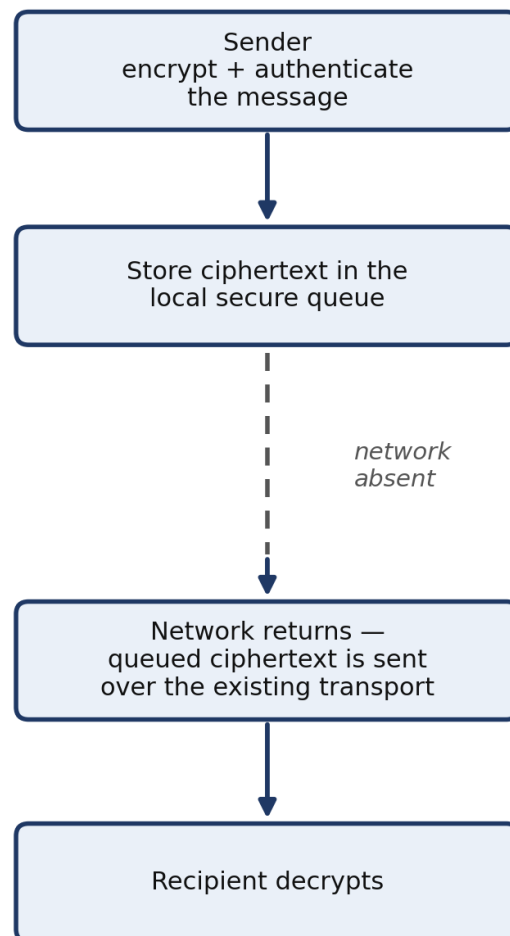
A standard browser sandbox does not permit arbitrary raw TCP sockets or multicast DNS discovery, both of which the LAN path requires. Aegis's demonstration client is therefore packaged as an Electron desktop application: the React UI runs as before, while the Electron main process provides the native networking capability the browser would otherwise block.



13. Disconnected / Degraded Operation

“Offline capability” is treated as three distinct, explicitly named conditions, because collapsing them into one loose idea invites exactly the kind of judge pushback the project needs to withstand:

Mode	Description	What happens
A — Connected	Normal internet path available	Client → Relay → Client, as in Section 8.
B — Locally connected, infrastructure unavailable	No internet, but both peers share a local network	Direct LAN path (Section 12) carries the same encrypted session.
C — Temporarily disconnected	Neither the internet nor a shared local network is reachable	Messages are encrypted and queued locally; delivery resumes once any transport becomes available again.



The encryption step is identical in all three modes. Only the delivery mechanism and timing change. This is the practical demonstration of the claim in Section 3: the security layer does not degrade because the network did.

14. What the Server Can and Cannot See

14.1 The blind-relay principle

The Spring Boot relay is deliberately simple. Its job is account authentication, session tokens, distributing public pre-key bundles, relaying ciphertext, and temporarily queuing encrypted messages for offline recipients. It is designed so that none of the information required to decrypt a message ever needs to reach it — this is meant as a structural guarantee, verifiable by inspection, not a policy promise the operator has to be trusted to honor.

The server may hold	The server must never hold
Account / user identifiers	Plaintext message content
Public identity keys and signatures	Private identity keys
Public pre-key bundles (X25519, ML-KEM)	Root keys
Encrypted message packets	Chain keys
Temporary offline-delivery queues	Message keys
Connection / session metadata	Any ratchet state

14.2 How this is demonstrated, not just claimed

The project's philosophy is to demonstrate security properties rather than assert them. For the blind-relay claim specifically: inspect the backend database directly during the demonstration. It should show account records, public keys, signatures, and ciphertext — and it should be possible to show, live, that no field in that database yields plaintext, private keys, or session secrets under any query.

14.3 The metadata caveat

“The server sees nothing” is false, and must never be claimed

Even a correctly-built blind relay still observes metadata: which accounts are communicating with which other accounts, message timing, packet frequency, approximate packet sizes, and IP/connection information, unless additional metadata-protection measures are explicitly implemented (which Aegis does not currently claim to have). Content confidentiality and metadata privacy are two different properties. Conflating them in front of judges is the single easiest way to lose credibility on this project.

14.4 Why not just TLS?

TLS secures the transport connection between a client and the server — it does not, by itself, prevent the server from reading application-level content, because the server is one endpoint of that TLS connection. If the architecture is Client -TLS- Server -TLS- Client, the server can terminate both connections and see plaintext in between. Aegis instead encrypts at the application layer, end-to-end, so that the packet the

server relays is already ciphertext it cannot open, independent of whatever transport security also happens to be present.

15. Threat Model

Nine adversary classes are considered explicitly. For each, the table states what the attacker can do and what property is expected to limit them.

Adversary	Capability	What limits them
Passive network observer	Observe traffic in transit	AES-256-GCM confidentiality; content is ciphertext regardless of path.
Active network attacker	Modify, inject, drop, or replay packets	AEAD authentication tags detect tampering; replay protection (Section 15.1) detects re-sent packets.
Malicious relay server	Attempts to inspect or manipulate traffic it relays	Blind-relay design (Section 14) — no plaintext or session secrets ever reach it.
Compromised relay server (data at rest)	Full read access to server storage	Same as above — stored data contains no decryption-relevant secrets, only ciphertext and metadata.
Malicious LAN peer	Present on the same local network	JmDNS provides no trust; identity verification (Section 7) is required before a session is treated as authentic.
Compromised endpoint (active)	Full control of a live client	Not mitigated while the compromise is active — this is a fundamental limit, stated openly rather than hidden (Section 9.3).
Stolen device (at rest)	Physical possession of a powered-off or locked device	Encrypted-at-rest local storage (Section 10), OS-backed key protection.
Long-term passive recorder (HNDL)	Stores ciphertext today for future decryption attempts	Post-quantum component of the hybrid handshake (Section 6).
Future quantum adversary	Would break X25519 alone via Shor's algorithm	ML-KEM component of the hybrid handshake is designed to remain secure independent of that break.

15.1 Replay protection specifics

Every packet carries a ratchet header including a chain index, so the receiver can uniquely identify which message key a given packet corresponds to. A packet reusing an already-consumed index is rejected. The receiver distinguishes a legitimately delayed packet (which arrives with an index it has not yet processed) from a replayed one (which reuses an index already processed and discarded) by tracking a bounded window of recently-seen indices per session.

15.2 What Aegis explicitly does not defend against

- An endpoint that is actively compromised while the compromise is ongoing (Section 9.3).
- Metadata correlation by a party observing traffic patterns over time (Section 14.3).

- Social-engineering or out-of-band compromise of the initial trust anchor if a user never performs any out-of-band verification (Section 7.4).
- Denial of service against relay availability — mitigated in effect by the LAN fallback path, but not a cryptographic guarantee.

16. Why These Specific Technologies

16.1 ML-KEM-768 over alternatives

ML-KEM (standardized by NIST as FIPS 203, derived from CRYSTALS-Kyber) was selected over other post-quantum KEM candidates such as Classic McEliece primarily on the basis of practical balance. Classic McEliece offers a long, conservative security track record but has public keys on the order of hundreds of kilobytes — impractical for a system explicitly targeting constrained, low-bandwidth links. ML-KEM-768 offers NIST Level 3 security with public keys and ciphertexts on the order of roughly 1.1–1.2 KB, a far better fit for the project's bandwidth constraints, and it is the primary standardized KEM most other production systems (including Signal's PQXDH) have converged on, which matters for implementation-ecosystem maturity and independent scrutiny. Any specific numeric comparison used in a presentation should be checked against the current NIST specification rather than quoted from memory.

16.2 X25519 as the classical component

X25519 is fast, has small keys, and has an exceptionally long, heavily-audited track record. It is the natural classical half of a hybrid construction precisely because its weaknesses (vulnerability to a future quantum computer) and ML-KEM's relative youth are complementary, not overlapping.

16.3 ML-DSA-65 for authentication

ML-DSA (FIPS 204, derived from CRYSTALS-Dilithium) was chosen to keep the entire trust chain — not just the key-establishment step — quantum-resistant. Using a classical signature scheme (e.g. Ed25519) to authenticate a post-quantum handshake would leave the authentication step itself vulnerable to the exact future adversary the rest of the system is designed to resist.

16.4 AES-256-GCM for payload encryption

Symmetric encryption is dramatically more efficient than public-key cryptography for bulk data, which is why Aegis follows the standard division: asymmetric/PQ cryptography establishes and authenticates keys, and symmetric cryptography does the actual work of encrypting messages. AES-256-GCM was chosen because it provides confidentiality and integrity/authentication in a single, hardware-accelerated, extremely well-studied primitive.

16.5 Spring Boot, Netty, and JmDNS

- Spring Boot — chosen for the relay because the relay's job (auth, key distribution, ciphertext relay, offline queueing) is deliberately simple, and a mature, well-documented framework minimizes the chance of introducing implementation bugs in code that is not the security-critical path.
- Netty — chosen for the LAN transport because it offers efficient, low-level, asynchronous TCP handling appropriate for a direct peer connection without pulling in unnecessary protocol logic.

- JmDNS — chosen because multicast DNS service discovery is a standard, widely supported mechanism for local peer discovery that requires no central coordination — consistent with the project's goal of working without infrastructure.

16.6 Electron for the demonstration client

As covered in Section 12.3, raw TCP sockets and multicast DNS are unavailable inside a standard browser sandbox. Electron was chosen over a fully native rewrite because it lets the same React codebase used for the UI also host the native networking layer, minimizing duplicated client logic between the internet and LAN code paths.

17. The Post-Quantum Packet-Size Problem

17.1 Why this is a real engineering problem

Post-quantum public keys, ciphertexts, and signatures are meaningfully larger than their classical counterparts. Representative sizes relevant to this project:

Object	Approximate size
ML-KEM-768 public key	≈ 1.2 KB
ML-KEM-768 ciphertext	≈ 1.1 KB
ML-DSA-65 signature	≈ 3.3 KB
AES-GCM authentication tag	16 bytes

These sizes are unremarkable on a 5G or broadband link. They are significant on the constrained, low-bandwidth, high-latency links the project's problem statement is actually about — for example, legacy 2G-class links in the tens of kilobits per second.

17.2 Mitigation strategy

1. Front-load the cost — the large ML-KEM/ML-DSA material is exchanged once, during session establishment, not attached to every ordinary message. After the handshake, ordinary traffic uses only the symmetric ratchet (Section 9), so PQC overhead is a setup cost rather than a per-message cost.
2. Binary serialization — Protobuf or CBOR is used in place of JSON-plus-Base64 for handshake and message framing. JSON/Base64 turns compact binary key material into larger text with encoding overhead; binary serialization avoids that overhead directly.
3. No blind compression — keys, ciphertexts, and signatures are high-entropy binary data and compress poorly. The mitigation is eliminating unnecessary encoding overhead, not gzip-ing already-dense secrets.
4. No silent downgrade — even under severe bandwidth or timeout pressure, the system does not fall back from the hybrid post-quantum handshake to a classical-only exchange. A slow network is handled with retry, chunked/resumable transfer, and queueing (Section 13), never by quietly removing the post-quantum protection.

17.3 Design targets versus measured facts

Numbers to state carefully

Prior research suggested roughly a one-third reduction in payload size from moving off JSON/Base64 encoding, and design targets of ordinary encrypted messages landing around 100–300 bytes with packets generally kept under ≈500 bytes. These are engineering estimates and design targets, not benchmark results. Any performance number presented to judges must be labeled as one of: a design target, an engineering estimate, or a measured benchmark — and only the third category may be called “measured.”

17.4 What must actually be benchmarked before presentation

- Handshake latency end-to-end, including network round-trip
- ML-KEM and ML-DSA operation time on representative hardware
- Actual serialized packet sizes (JSON/Base64 baseline vs. binary serialization)
- Memory and CPU usage under sustained messaging
- Behavior and throughput under constrained/simulated bandwidth
- Reconnection and handover latency (Section 11.2)

18. Deployment Scenarios

Aegis is described here in terms of the environments it is designed for, using technically defensible language rather than absolute claims. The following phrase is the one used consistently throughout this document and any presentation of the project:

The defensible framing

“Designed to provide confidentiality and authentication against classical and currently anticipated quantum adversaries, under the security assumptions of the selected standardized primitives and the correctness of the protocol implementation.” Terms such as ‘unhackable,’ ‘100% secure,’ ‘quantum-proof,’ ‘impossible to decrypt,’ or ‘military-grade therefore secure’ are not used anywhere in this project's materials.

18.1 Illustrative environments

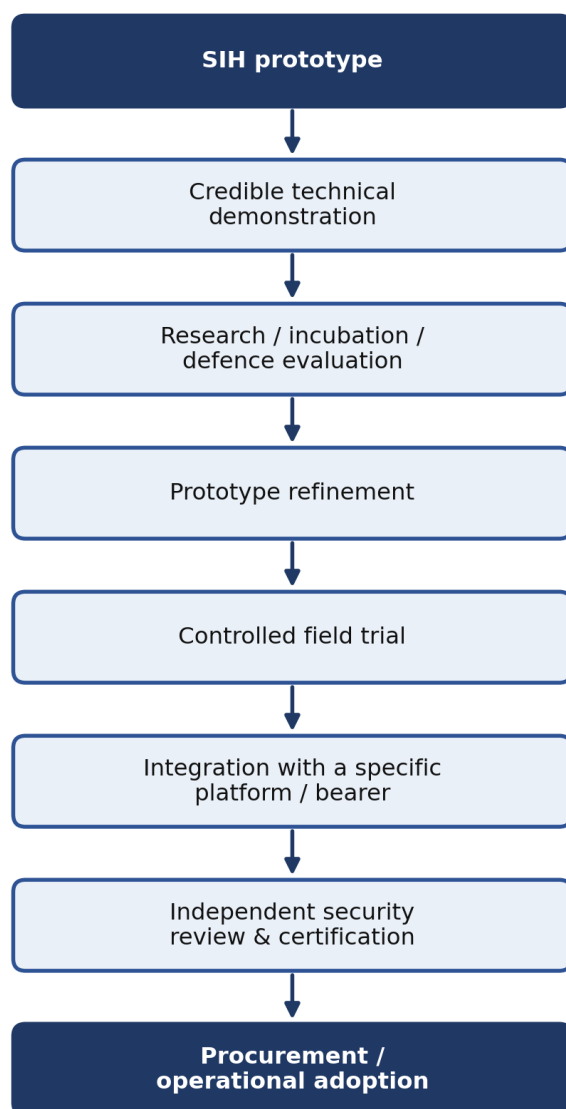
Environment	Relevant Aegis property
Disaster response / temporary command posts	Continued operation over LAN once internet infrastructure is damaged or absent (Section 12–13).
Rural / remote-area coordination	Same LAN fallback; low-bandwidth packet design for degraded links (Section 17).
Long-confidentiality communication (government, legal, medical, defense-adjacent)	Hybrid post-quantum handshake defends against harvest-now-decrypt-later (Section 6, Section 2.1).
Organizations wary of centralized trust	Blind-relay architecture limits what a compromised or coerced server can expose (Section 14).
Constrained tactical-style environments (as a research direction, not a certified deployment)	Transport abstraction (Section 11) that could, in future work, be extended to additional bearer types beyond TCP and LAN.

18.2 Sovereignty and indigenous-development positioning

Aegis can reasonably be discussed in the context of cybersecurity self-reliance, but not on the simplistic basis that “foreign libraries are insecure.” The stronger and more accurate argument is that critical systems benefit from auditable, independently controlled, reproducible implementations and a minimized supply-chain footprint. This is a distinct claim from the algorithms themselves: ML-KEM, X25519, ML-DSA, and AES-GCM are internationally standardized primitives, and using standardized algorithms does not make a system any less sovereign — what determines sovereignty is who controls the implementation, the deployment infrastructure, and the audit process, not which country published the underlying mathematics.

18.3 The realistic path from prototype to deployment

This is stated plainly because judges will ask, and an overclaimed answer is worse than an honest one:



A six-month hackathon prototype occupies the first step of this chain. It does not, and should not claim to, occupy the last one.

19. Known Gaps and Roadmap

Treating unresolved items honestly is itself part of the engineering discipline this project is built around. This section distinguishes established project direction from work still to be done, using the classification the team applies throughout its own research process:

Category	Meaning
Established project direction	Explicitly selected as part of the architecture.
Engineering proposal	A suggested way to implement part of the architecture, not yet built.
Benchmark result	Something actually measured.
Security claim	Something requiring cryptographic reasoning or evidence, not assertion.
Hypothesis	Something that still needs validation.
Future scope	Not yet implemented; a direction for later work.

The rule that follows from this: “we plan to” is never rewritten as “we have implemented,” “estimated” is never rewritten as “measured,” and “should be secure” is never rewritten as “is proven secure.”

19.1 Open items, tracked explicitly

Open item	Current status
TOFU + fingerprint/QR identity verification	Design decided (Section 7.4); implementation and UX not yet built.
Encrypted-at-rest ratchet state with atomic persistence	Design decided (Section 10); implementation and crash-safety testing not yet complete.
Automatic, seamless transport handover	Cryptographic compatibility across transports is architected (Section 11); the failure-detection and switching logic itself still needs to be built and stress-tested.
One-time pre-key exhaustion / replenishment protocol	Fallback behavior is specified (fall back to the signed pre-key, still run the full hybrid handshake, replenish on reconnect) but not yet implemented end-to-end.
5-second-plus handshake timeout / resumable transfer	No timeout/fallback protocol is implemented yet; sensible behavior is proposed (retry with backoff, then attempt LAN discovery, then queue) but not built.
Additional simulated/tactical bearer beyond TCP and LAN	Future scope; not part of the current prototype.
Formal protocol verification / independent cryptographic audit	Not performed. Required before any claim stronger than ‘prototype’ would be appropriate.

19.2 Implementation phases

1. Cryptographic proof of concept — ML-KEM, X25519, hybrid derivation, symmetric encryption.

2. Authentication — ML-DSA identity and pre-key authentication.
3. Double Ratchet — root/chain/message key state, DH ratchet, skipped-message handling.
4. Message protocol — packet format, message IDs, headers, acknowledgements, replay handling, ordering, retransmission.
5. Blind relay — Spring Boot backend functionality.
6. LAN — JmDNS discovery and Netty transport using the identical encrypted packet format.
7. Failure testing — server failure, internet failure, LAN-only operation, packet loss/duplication/reordering, offline recipients, invalid signatures/ciphertext, malicious relay and LAN peer behavior.
8. Benchmarking — handshake latency, per-operation timing, packet sizes, memory/CPU, throughput under constrained bandwidth, reconnection behavior.

19.3 Future scope beyond the prototype

- Secure voice and richer media types over the same overlay.
- Additional simulated or real constrained bearers (tactical-style links, SATCOM-adjacent simulations).
- A lightweight operational identity/provisioning model — a command authority that provisions and can revoke device identities, without attempting to reproduce a full military PKI (Section 3.3).
- Disconnected/mesh-style multi-hop relaying beyond direct two-party LAN contact.

None of the items in this subsection are presented as implemented. They are the roadmap, not the deliverable.

20. Judge FAQ

Straight answers to the questions this project should expect, phrased the way the team intends to answer them out loud.

Why should we not just use Signal?

We aren't competing with Signal as a messaging application — Signal already solves secure messaging extremely well for its target environment. Our contribution is a cryptographic communications layer that stays secure and operational across multiple transport conditions, including a direct local-network path used when no central infrastructure is reachable at all. The chat interface is a demonstration client for that layer, not the product.

PQCChat already does post-quantum messaging. What's novel here?

PQCChat demonstrates PQC messaging over the internet, which is a real and useful thing to have built. Our focus is different: transport-independent operation and degraded/disconnected behavior, with the cryptographic layer deliberately decoupled from whatever bearer is carrying it. We show this by running the identical session over more than one transport.

What exactly makes this suitable for a military or defense context?

Not the algorithms by themselves — ML-KEM and AES-GCM are not 'military' technologies. What's relevant is the set of constraints the prototype is built around: intermittent connectivity, constrained bandwidth, delayed delivery, device loss, controlled identity provisioning, and operation without a continuously available central server. Those are the actual constraints of the target environment, addressed directly rather than assumed away.

Which military radio or classified network does this integrate with?

None, and we do not claim otherwise. The prototype defines a transport interface and demonstrates it over generic bearers — TCP, a direct LAN connection, and a simulated constrained link. Integrating a specific fielded radio or classified network would be a subsequent engineering phase, well beyond a six-month student prototype, and we say that plainly rather than implying otherwise.

Why ML-KEM-768 specifically?

It is NIST's standardized post-quantum KEM (FIPS 203), it offers a practical balance of key/ciphertext size versus security level compared to alternatives like Classic McEliece (which has far larger keys), and it is the primary post-quantum KEM other production systems, including Signal's PQXDH, have converged on — which matters for implementation maturity and independent scrutiny.

What mathematical problem protects ML-KEM?

Hardness assumptions from Module Learning With Errors, a lattice-based problem. This is a different mathematical foundation from RSA (integer factorization) or elliptic-curve cryptography (discrete logarithms), both of which Shor's algorithm attacks efficiently on a sufficiently capable quantum computer.

No known quantum algorithm currently gives an analogous practical break against the lattice assumptions ML-KEM relies on.

Why combine X25519 and ML-KEM instead of just using one?

Defense in depth. If ML-KEM ever turns out to have an unforeseen weakness — post-quantum cryptography is comparatively young — X25519 still contributes real classical security. If a quantum computer eventually breaks X25519, ML-KEM is designed to remain resistant under its own assumptions. Neither is trusted in isolation.

Why ML-DSA specifically, and not a classical signature scheme?

Because using a classical signature to authenticate a post-quantum handshake would leave the authentication step itself exposed to the same future quantum adversary the rest of the design defends against. ML-DSA keeps the whole trust chain, not just key establishment, quantum-resistant.

Why the Double Ratchet?

It gives forward secrecy (compromising a current key doesn't expose past messages, since past keys are already discarded) and post-compromise recovery (the periodic DH step reintroduces fresh secret material after a temporary compromise ends). We do not claim to have invented it — it's a well-studied construction — our contribution is composing it correctly inside a transport-independent architecture.

Why not just use TLS?

TLS secures a transport connection between a client and whichever server terminates it — it does not prevent that server from reading application-level content, since the server is one endpoint of the TLS session. Aegis encrypts at the application layer, end-to-end, so the relay only ever sees ciphertext it cannot open, regardless of what transport security also happens to be in place underneath.

What happens if the relay server is fully compromised?

The attacker obtains ciphertext, public key material, and metadata (Section 14.3) — not the secrets required to decrypt message content, because those secrets never reach the server by design. We can show this by inspecting the server's storage directly.

What happens if a client device is compromised?

While the compromise is active, the attacker has the same access the legitimate user has — no architecture can fully prevent that, and we say so directly rather than overclaiming. What the ratchet does provide is a bound on how long that exposure persists: past messages stay protected by forward secrecy, and future security can recover once the compromise ends and the ratchet advances again.

PQC keys are large. How do you handle low-bandwidth links?

Three things: the expensive PQC material is exchanged once at handshake time, not per message; ordinary messages use binary serialization instead of JSON/Base64 to cut encoding overhead; and we never silently

downgrade to classical-only cryptography just because a network is slow. All performance numbers we present are labeled as design targets, estimates, or measured benchmarks — never blurred together.

Can it really work with zero internet?

Yes, when both devices share a local network path — demonstrated via the direct LAN transport. Full isolation with no local network at all falls back to local encrypted queueing until any transport becomes available again (Section 13).

How do you authenticate a peer discovered over the LAN?

JmDNS only provides discovery — finding a device, not verifying it. Cryptographic identity verification (ML-DSA signatures plus the TOFU/fingerprint trust model in Section 7) is what actually authenticates the peer, entirely independent of how it was found on the network.

Can the server read the messages?

No — by design, the server never receives the private keys, root key, chain keys, or message keys required to decrypt content. It can be shown, by inspecting its storage, to hold only public keys, signatures, ciphertext, and metadata.

Is this actually going to be deployed by [X]?

Not directly from this prototype. The realistic path from a hackathon prototype to operational deployment runs through independent security review, protocol auditing, extensive testing, and procurement — a multi-year institutional process (Section 18.3). What we can defend today is that the underlying engineering problem is real, the primitives are sound and standardized, and the prototype demonstrates the core architectural claims honestly.

21. Glossary

Term	Meaning
AEAD	Authenticated Encryption with Associated Data — encryption that provides both confidentiality and integrity/authentication in one operation, e.g. AES-256-GCM.
AES-256-GCM	The symmetric cipher used to encrypt actual message content once a session is established.
Double Ratchet	The mechanism that continuously derives fresh per-message keys and periodically refreshes the root key, providing forward secrecy and post-compromise recovery.
Forward secrecy	The property that compromising current key material does not expose previously sent messages, because old keys have already been discarded.
Harvest Now, Decrypt Later (HNDL)	An attack strategy where an adversary records encrypted traffic today with the intent to decrypt it once sufficiently powerful quantum computers exist.
HKDF-SHA-256	HMAC-based key derivation function used to turn combined raw secret material into a structured root key and further session keys.
Hybrid handshake	A key-establishment step combining a classical mechanism (X25519) and a post-quantum mechanism (ML-KEM) so that breaking either one alone is insufficient to recover the session secret.
JmDNS	A Java implementation of multicast DNS service discovery, used for local peer discovery on a LAN. Provides discovery only, not authentication.
ML-DSA-65	A standardized (FIPS 204) post-quantum digital signature algorithm, used to authenticate identity and pre-key bundles.
ML-KEM-768	A standardized (FIPS 203) post-quantum key encapsulation mechanism based on lattice problems (Module-LWE), used for post-quantum key establishment.
Netty	An asynchronous network application framework used for the direct TCP transport between LAN peers.
Post-compromise recovery	The property that a session can regain confidentiality after a temporary compromise ends, once fresh secret material is reintroduced by the DH ratchet step.
Pre-key bundle	A signed collection of public key material a user publishes in advance, allowing another user to start a secure session even while the recipient is offline.
Root key	The top-level session secret derived from the hybrid handshake, from which the Double Ratchet's chain keys and message keys are ultimately derived.
Trust On First Use (TOFU)	A trust model where an identity key is accepted the first time it is seen and any later change to that key triggers a warning, rather than requiring prior out-of-band verification for every contact.
X25519	A classical elliptic-curve Diffie-Hellman key agreement function, providing the classical half of the hybrid handshake.

22. Team & Contributors

Aegis is being developed for Smart India Hackathon 2026 by the team below. Replace the placeholder rows with your team's actual details before submission.

22.1 Team details

Field	Details
Team name	QUBIT
Problem statement ID	SIH26237
Problem statement title	Post Quantum Secure Communication Overlay
Theme	CyberSecurity
Category	Software
Institution	VU